

```
# =====
```

```
# MACHINE LEARNING MASTER CODE
```

```
# =====
```

```
# EXPERIMENTS:
```

```
# 1. Retail Sales - EDA & Data Preprocessing
```

```
# 2. House Price Prediction - Linear Regression
```

```
# 3. Loan Application - Logistic Regression
```

```
# 4. Credit Risk Assessment - Random Forest
```

```
# 6. Customer Churn - Decision Tree
```

```
# 7. Email Spam Detection - Naive Bayes
```

```
# =====
```

```
# MEMORIZE THE COMMON PART
```

```
# =====
```

```
# COMMON FLOW:
```

```
# IMPORT → LOAD → EXPLORE → CLEAN → X,y → SPLIT
```

```
# → MODEL → FIT → PREDICT → EVALUATE
```

```
# ONLY CHANGE:
```

```
# 1. IMPORT STATEMENT
```

```
# 2. DATA PREPROCESSING
```

```
# 3. MODEL NAME
```

```
# 4. EVALUATION METRICS
```

```
# =====
```

```
# 1. COMMON LIBRARIES
```

```
# =====
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# =====
```

```
# 2. LOAD DATA
```

```
# =====
```

```
df = pd.read_csv("your_dataset.csv")
```

```
print(df.head())
print(df.info())
print(df.describe())
```

```
# =====
```

```
# 3. DATA CLEANING
```

```
# =====
```

```
# Check missing values
print(df.isnull().sum())
```

```
# Remove missing values
df = df.dropna()
```

```
# Remove duplicate values
```

```
df = df.drop_duplicates()
```

```
# =====
```

```
# =====
```

```
# EXPERIMENT 1 : RETAIL SALES - EDA
```

```
# =====
```

```
# =====
```

```
# NO ML MODEL IS REQUIRED
```

```
# Perform EDA and preprocessing
```

```
# Statistical Analysis
```

```
print(df.mean(numeric_only=True))
```

```
print(df.var(numeric_only=True))
```

```
print(df.std(numeric_only=True))
```

```
# Covariance
```

```
print(df.cov(numeric_only=True))
```

```
# Correlation
```

```
print(df.corr(numeric_only=True))
```

```
# -----
```

```
# NORMALIZATION
```

```
# -----
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
numeric_cols = df.select_dtypes(include=np.number).columns
```

```
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

```
# -----
```

```
# STANDARDIZATION
```

```
# -----
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

```
# -----
```

```
# DISCRETIZATION
```

```
# -----
```

```
df["category"] = pd.cut(
```

```

df[numeric_cols[0]],
bins=3,
labels=["Low", "Medium", "High"]
)

print(df.head())

# -----
# OUTLIER DETECTION USING IQR
# -----

Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)

IQR = Q3 - Q1

df = df[
    ~(
        (df[numeric_cols] < (Q1 - 1.5 * IQR)) |
        (df[numeric_cols] > (Q3 + 1.5 * IQR))
    ).any(axis=1)
]

# -----
# VISUALIZATION
# -----

```

```
# Histogram
```

```
sns.histplot(df[numeric_cols[0]], kde=True)
```

```
plt.show()
```

```
# Boxplot
```

```
sns.boxplot(x=df[numeric_cols[0]])
```

```
plt.show()
```

```
# Scatter plot
```

```
sns.scatterplot(
```

```
    x=df[numeric_cols[0]],
```

```
    y=df[numeric_cols[1]]
```

```
)
```

```
plt.show()
```

```
# Correlation Heatmap
```

```
sns.heatmap(
```

```
    df.corr(numeric_only=True),
```

```
    annot=True
```

```
)
```

```
plt.show()
```

```
# =====
```

```
# FROM HERE = COMMON ML CODE
```

```
# =====
```

```
#
```

Use this structure for:

2. Linear Regression

3. Logistic Regression

4. Random Forest

6. Decision Tree

7. Naive Bayes

#

BUT:

Assignment 7 has TEXT preprocessing + TF-IDF.

=====

=====

EXPERIMENT 2 : LINEAR REGRESSION

HOUSE PRICE PREDICTION

=====

IMPORT

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import (

mean_absolute_error,

mean_squared_error,

r2_score

```
)
```

```
# -----
```

```
# LOAD DATA
```

```
# -----
```

```
df = pd.read_csv("house_price.csv")
```

```
print(df.head())
```

```
# -----
```

```
# DEFINE X AND y
```

```
# -----
```

```
X = df[
```

```
[
```

```
    'area',
```

```
    'bedrooms',
```

```
    'bathrooms',
```

```
    'parking',
```

```
    'age'
```

```
]
```

```
]
```

```
y = df['price']
```



```
# -----
```

```
# SPLIT DATA
```

```
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
# -----
```

```
# CREATE MODEL
```

```
# -----
```

```
model = LinearRegression()
```

```
# -----
```

```
# TRAIN
```

```
# -----
```

```
model.fit(X_train, y_train)
```

```
# -----
```

```
# PREDICT
```

```
# -----
```

```
y_pred = model.predict(X_test)
```

```
# -----
```

```
# EVALUATION
```

```
# -----
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("MAE :", mae)
```

```
print("MSE :", mse)
```

```
print("RMSE :", rmse)
```

```
print("R2 Score :", r2)
```